

RHCSA EX200 — Resumo Dia 04/05

Usuários · Grupos · Permissões · ACL · Umask · Sudo

PROVA: 14/05

⚠️ Armadilhas do Dia

Certo	Errado
<code>usermod -aG grupo user</code>	sem <code>-a</code> remove TODOS os outros grupos
<code>chmod 2770</code> (com o 2)	sem o 2 não há SGID
<code>setfacl -m d:grupo:rx</code>	sem <code>d:</code> novos arquivos ignoram a ACL
umask no <code>.bashrc</code>	umask no terminal some ao fechar a sessão
<code>NOPASSWD: /usr/bin/dnf</code>	caminho relativo <code>dnf</code> é inseguro

📊 Tabela chmod Especial

Dígito	Nome	Efeito
4xxx	SUID	Executável roda com permissão do dono
2xxx	SGID	Diretório: novos arquivos herdam o grupo
1xxx	Sticky	Só o dono do arquivo pode deletá-lo

BLOCO 1 — Usuários e Grupos

Flags essenciais do `useradd` :

- `-u` `UID` — UID fixo. Sem isso o sistema escolhe o próximo disponível.
- `-s` `SHELL` — Shell do usuário. `/sbin/nologin` rejeita conexões interativas — contas de serviço.
- `-e` `DATA` — Data de expiração. Formato obrigatório: `YYYY-MM-DD` .
- `-G` `GRUPOS` — Grupos secundários. Vírgula separa múltiplos: `-G g1,g2` .

```
useradd alex                                # usuário simples
useradd -u 1500 -s /sbin/nologin sam        # UID fixo + conta de serviço
useradd -u 1250 -e 2029-12-21 john         # UID + expiração
useradd -G developers alex                 # grupo secundário
useradd -G accounting,finance alex         # múltiplos grupos

usermod -s /bin/bash sam                   # mudar shell
usermod -aG developers alex                # ADICIONAR grupo (mantém outros)
usermod -u 2000 alex                       # mudar UID

groupadd developers

# Verificação
id alex
grep alex /etc/passwd
chage -l john
su - harry      # testar nologin
```

Exemplo completo (S3-Q7): mudar shell de `sam` para bash + criar `john` com UID 1250 e expiração

```
usermod -s /bin/bash sam
useradd -u 1250 -e 2029-12-21 john
```

`usermod -G` (sem `-a`) **substitui** todos os grupos secundários. Sempre use `-aG` para adicionar sem remover os existentes. Formato de data obrigatório: `YYYY-MM-DD` .

BLOCO 2 — Políticas Globais

Quatro arquivos, quatro funções completamente diferentes:

/etc/skel — Template do home

Tudo aqui é copiado automaticamente para o home de **novos** usuários. Não afeta usuários já existentes.

Exemplo (S1-Q12):

```
touch /etc/skel/Welcome.txt
useradd testuser123
ls /home/testuser123/      # o arquivo deve aparecer aqui
```

/etc/login.defs — Expiração de senha

`PASS_MAX_DAYS` define o máximo de dias que uma senha pode ser usada. Afeta apenas usuários criados **depois** da alteração. Para usuários existentes: `chage -M 90 username` .

Exemplo (S1-Q12):

```
vim /etc/login.defs
# PASS_MAX_DAYS 90
```

/etc/security/pwquality.conf — Complexidade de senha

`minlen` define o número mínimo de caracteres.

Exemplo (S1-Q12):

```
vim /etc/security/pwquality.conf
# minlen = 8
```

/etc/profile.d/ — Variáveis globais

Qualquer arquivo `.sh` aqui roda no login de **todos** os usuários — mais seguro que editar `/etc/profile` direto.

Exemplo completo (S6-Q28):

```
vim /etc/profile.d/custom_env.sh
# export VAR="RHCSA"
# export HISTFILESIZE=2000
source /etc/profile.d/custom_env.sh
echo $VAR           # RHCSA
echo $HISTFILESIZE   # 2000
```

BLOCO 3 — Permissões Especiais

SGID em diretório (chmod 2xxx)

Arquivos criados dentro herdam o **grupo do diretório**, não o grupo primário do criador. Essencial para colaboração em equipe.

Exemplo completo (S1-Q13): grupo `developers`, `/opt/dev-data` com SGID, owner+group=RWX, others sem acesso

```
groupadd developers
mkdir /opt/dev-data
chown :developers /opt/dev-data      # grupo do diretório
chmod 2770 /opt/dev-data             # 2=SGID | 7=owner rwx | 7=group rwx | 0=others
ls -ld /opt/dev-data
# drwxrws--- (o 's' no lugar do 'x' do grupo = SGID ativo)
```

`chmod 2770` vs `chmod 770` : sem o `2` não há SGID e arquivos novos herdam o grupo primário do criador, não o grupo do diretório. Sempre `chown :grupo` antes do `chmod 2xxx` .

Sticky Bit (chmod 1xxx)

Cada usuário só pode deletar os próprios arquivos no diretório (resolve o problema do `/tmp`).

Exemplo completo (S3-Q23): `/data/shared` , permissão 777, sticky bit

```
mkdir -p /data/shared
chmod 1777 /data/shared             # 1=sticky | 777=todos rwx
ls -ld /data/shared
# drwxrwxrwt (o 't' no lugar do 'x' de others = sticky ativo)
```

Combinação avançada — SGID + ACL cross-group (S5-Q14/S6-Q17):

```
groupadd Innovation && groupadd Solution
mkdir -p /groups/Innovation /groups/Solution
chown :Innovation /groups/Innovation
chown :Solution /groups/Solution
chmod 2770 /groups/Innovation /groups/Solution

setfacl -m g:Solution:rx /groups/Innovation      # acesso atual
setfacl -m d:g:Solution:rx /groups/Innovation    # herança futura
getfacl /groups/Innovation
```

São **dois** `setfacl` : sem `d:` (diretório agora) e com `d:` (arquivos futuros). Esquecer o `d:` é o erro mais comum.

BLOCO 4 — ACL (Access Control Lists)

Quando usar **ACL vs chmod**: `chmod` trabalha só com owner/group/others. ACL permite permissões para usuários/grupos específicos além dessas três.

```
# setfacl -m TIPO:NOME:PERMS arquivo | TIPO: u (user) | g (group) | d (default/herança)
setfacl -m u:alex:rw /arquivo        # alex: leitura + escrita
setfacl -m u:kevin:--- /arquivo      # kevin: SEM acesso
setfacl -m g:finance:rx /diretorio   # grupo: leitura + execução
setfacl -m d:g:finance:rx /diretorio # DEFAULT ACL — herança para novos arquivos

getfacl /arquivo                     # lista todas as ACLs
ls -l /arquivo                       # '+' no final = tem ACL

setfacl -x u:alex /arquivo           # remover ACL do alex
setfacl -b /arquivo                  # remover TODAS as ACLs
```

Exemplo completo (S2-Q15): `/etc/fstab` → `/var/tmp/fstab` , owner+group=root, remover execute de todos, `stewart` =RW, `kevin` =sem acesso

```
cp /etc/fstab /var/tmp/fstab
chown root:root /var/tmp/fstab
chmod a-x /var/tmp/fstab             # remove execute de owner+group+others
setfacl -m u:stewart:rw /var/tmp/fstab
setfacl -m u:kevin:--- /var/tmp/fstab
getfacl /var/tmp/fstab
```

O `+` no `ls -l` indica ACL ativa. A `mask` no `getfacl` é o teto máximo para qualquer ACL (exceto owner).

BLOCO 5 — Umask

Umask define as permissões **removidas** dos valores base: arquivo base=666 (sem execute), diretório base=777.

Umask	Arquivo (cálculo)	Arquivo	Dir	Uso típico
022	666-022=644	rw-r--r--	755 rwxr-xr-x	Padrão do sistema
027	666-027=640	rw-r-----	750 rwxr-x---	Segurança moderada
077	666-077=600	rw-----	700 rwx-----	Máxima privacidade
002	666-002=664	rw-rw-r--	775 rwxrwxr-x	Colaboração em grupo

Exemplo completo (S1-Q28): umask 027 para `harry`, persistente (640/750)

```
vim /home/harry/.bashrc
# umask 027

su - harry
umask                # 0027
touch testfile && ls -l testfile  # -rw-r----- (640)
mkdir testdir && ls -ld testdir   # drwxr-x--- (750)
```

`.bashrc` roda em todo shell interativo. `.bash_profile` só no login inicial. Umask no terminal = temporário; no `.bashrc` = permanente.

BLOCO 6 — Sudo (sudoers.d) + Links Hard e Soft

Sudo com /etc/sudoers.d/

Arquivos aqui são incluídos automaticamente no sudo. Um erro de sintaxe não quebra o sudo inteiro — mais seguro que editar `/etc/sudoers` direto.

Exemplo completo (S1-Q19):

```
useradd alex
vim /etc/sudoers.d/alex
# alex ALL=(ALL) NOPASSWD: /usr/bin/dnf
#                               ^^^^^^^^^^^^^^ CAMINHO ABSOLUTO obrigatório

which dnf                # confirma o caminho: /usr/bin/dnf

su - alex
sudo dnf repolist        # deve funcionar SEM senha
sudo reboot              # deve ser negado ou pedir senha
```

Sempre caminho absoluto. Caminho relativo (`dnf`) é inseguro — manipulação do PATH poderia enganar o comando.

Links Hard e Soft

Hard link: dois nomes apontando para o **mesmo inode**. Soft link: um ponteiro para o **caminho** de outro arquivo.

Exemplo completo (S2-Q24):

```
touch /home/root/data.txt

ln /home/root/data.txt /home/root/data-hard      # HARD LINK (sem -s)
ln -s /home/root/data.txt /var/tmp/data-soft     # SOFT LINK (com -s)

ls -li /home/root/data*        # hard links têm o MESMO número de inode
ls -l /var/tmp/data-soft       # data-soft -> /home/root/data.txt
```

	Hard Link	Soft Link
Flag	<code>ln</code> (sem flag)	<code>ln -s</code>
O que armazena	Mesmo inode	Caminho do alvo
Se original é deletado	Link continua funcionando	Link quebra
Entre filesystems	Não funciona	Funciona
Para diretórios	Não permitido	Permitido

Tabela Comparativa — Erros Comuns

Situação	Correto ✓	Errado ✗
Adicionar grupo	<code>usermod -aG grupo user</code>	<code>usermod -G grupo user</code> (remove outros!)
SGID em diretório	<code>chmod 2770 dir</code> + <code>chown :grupo</code>	Esquecer o <code>chown :grupo</code>
ACL negar acesso	<code>setfacl -m u:user:--- arq</code>	Não criar a ACL (others acessa)
Default ACL	<code>setfacl -m d:g:grupo:rx dir</code>	Só <code>setfacl</code> sem <code>d:</code> (novos ignoram)
Umask persistente	<code>echo 'umask 027' >> ~/.bashrc</code>	<code>umask 027</code> no terminal (só a sessão)
Sudo sem senha	<code>NOPASSWD: /usr/bin/dnf</code>	<code>NOPASSWD: dnf</code> (caminho relativo)

Checklist Final

Usuários e Grupos

- `useradd` com `-u` (UID), `-s` (shell), `-e` (expiração), `-G` (grupos)
- `usermod -aG` para adicionar grupo SEM remover os outros
- Verificar com `id`, `grep /etc/passwd`, `chage -l`

Políticas Globais

- `touch /etc/skel/arquivo` → aparece no home de novos usuários
- `/etc/login.defs` → `PASS_MAX_DAYS` para expiração de senha
- `/etc/security/pwquality.conf` → `minlen` para tamanho mínimo
- `/etc/profile.d/custom.sh` → variáveis globais para todos

Permissões Especiais

- `chmod 2770` = SGID + owner rwx + group rwx + others ---
- `chmod 1777` = Sticky + todos rwx
- `ls -ld` mostra `s` no group = SGID ativo, `t` em others = Sticky ativo
- SEMPRE `chown :grupo` antes do `chmod 2xxx`

ACL

- `setfacl -m u:user:rw arquivo` (permissão por usuário)
- `setfacl -m u:user:--- arquivo` (negar acesso via ACL)
- `setfacl -m d:g:grupo:rx dir` (Default ACL = herança)
- `getfacl` confirma ACLs; `ls -l` mostra `+` quando há ACL

Umask

- `umask 027` → arquivo 640 | diretório 750
- `umask 077` → arquivo 600 | diretório 700 (máxima privacidade)
- umask no `.bashrc` para persistência por usuário

Sudo e Links

- `/etc/sudoers.d/user` com caminho ABSOLUTO do comando
- `NOPASSWD: /usr/bin/dnf` (não `NOPASSWD: dnf`)
- `ln` = hard link (mesmo inode) | `ln -s` = soft link (ponteiro para caminho)